

Masterwork & Special Materials System — Architecture

Design Principles

1. **Central Material Properties** — All material stats live in one place (`MaterialProperties.cs`). No item definition anywhere in the codebase stores hardcoded material bonuses.
 2. **Dynamic Runtime Calculations** — Every bonus (attack, damage, ACP, max dex, weight, spell failure) is computed at access time from the `ItemMaterial` attached to the item, not baked into static fields.
 3. **Factory Pattern** — `ItemMaterialFactory` generates all material variant items at initialization. A convenience `ApplyMaterial()` method allows runtime application to any item.
 4. **Extensibility** — Adding a new material requires: (a) add enum value to `ItemMaterialType` , (b) add properties in `MaterialProperties` , (c) optionally add to factory registration lists. Zero changes to item definitions, combat code, or UI.
-

File Map

```
Assets/Scripts/Equipment/
[ ] ItemMaterial.cs           # Enum + data class
[ ] MaterialProperties.cs     # Central stats database (single source of truth)
[ ] ItemMaterialFactory.cs   # Variant creation, registration, loot helpers
[ ] MATERIAL_SYSTEM_ARCHITECTURE.md # This file

Assets/Scripts/Inventory/
[ ] ItemData.cs              # Runtime-computed Effective* properties
[ ] ItemDatabase.cs          # RegisterItem(), calls factory at init
[ ] Inventory.cs             # Equip logic reads Effective* properties

Assets/Scripts/Character/
[ ] CharacterController.cs    # MasterworkAttackBonus + MaterialDamageModifier in
combat
[ ] CharacterStats.cs         # Mithral category shift for proficiency/speed/ACP
```

Data Flow

```
MaterialProperties.GetWeaponMaterial(type, baseItem)
```



```
ItemMaterial {  
    MaterialType, WeaponBypassTags, DamageModifier,  
    WeightMultiplier, AdditionalCostGp, ...  
}
```



```
ItemData.Material = itemMaterial
```



```
ItemData.MasterworkAttackBonus
```

+1 **if** MW && no magic enhancement



```
ItemData.MaterialDamageModifier
```

Material.DamageModifier (-1 **for** sil-

ver)



```
ItemData.EffectiveArmorCheckPenalty
```

base - MW reduction - material re-

duction



```
ItemData.EffectiveMaxDexBonus
```

base + material increase



```
ItemData.EffectiveArcaneSpellFailure
```

base - material reduction



```
ItemData.EffectiveWeightLbs
```

base ☒ material multiplier



```
ItemData.FullDisplayName
```

"Adamantine Masterwork Longsword +1"



```
ItemData.GetBypassTags()
```

Material.WeaponBypassTags | legacy f

lags

Supported Materials (D&D 3.5e PHB/DMG)

Material	Weapon Effects	Armor Effects	Cost Formula
Adamantine	Bypass DR/adamantine	DR 1/— (light), 2/— (medium), 3/— (heavy)	Weapon +3000gp, Armor +5000/10000/15000
Mithral	Half weight	Half weight, -1 category, +2 max dex, -3 ACP, -10% ASF	+1000/4000/9000 by category
Cold Iron	Bypass DR/cold iron	N/A	Double base weapon price
Alchemical Silver	Bypass DR/silver, -1 damage	N/A	+20 ammo, +90 light, +3000 1H, +9000 2H
Darkwood	Half weight (wooden only)	Half weight (wooden shields)	+10gp per lb saved

How to Add a New Material

Step 1: Add Enum Value

```
// ItemMaterial.cs
public enum ItemMaterialType
{
    Standard, Adamantine, Mithral, ColdIron, AlchemicalSilver, Darkwood,
    NewMaterial // ← Add here
}
```

Step 2: Define Properties in MaterialProperties.cs

```
// In GetWeaponMaterial() switch:
case ItemMaterialType.NewMaterial:
    mat.WeaponBypassTags = DamageBypassTag.NewTag; // if applicable
    mat.DamageModifier = 0;
    mat.WeightMultiplier = 1.0f;
    mat.AdditionalCostGp = 1000;
    break;

// In GetArmorMaterial() switch (if applicable):
case ItemMaterialType.NewMaterial:
    mat.ArmorCheckPenaltyReduction = 1;
    mat.MaxDexBonusIncrease = 1;
    mat.AdditionalCostGp = 2000;
    break;
```

Step 3: Add Validation (if needed)

```
// In IsMaterialValidForItem():
case ItemMaterialType.NewMaterial:
    return item.IsWeapon; // restrict to weapons only
```

Step 4: Register Variants (optional — for pre-built catalog)

```
// In RegisterAllMaterialVariants():
RegisterVariant(CreateMaterialWeapon(baseWeapon, ItemMaterialType.NewMaterial), ref count);
```

Step 5: Add Display Prefix

```
// In GetMaterialPrefix():
case ItemMaterialType.NewMaterial: return "New Material";
```

That's it. No changes needed to:

- ItemData (runtime properties auto-compute)
 - CharacterController (reads from ItemData properties)
 - CharacterStats (reads from ItemData properties)
 - Inventory (reads Effective* properties)
 - Any UI code (reads FullDisplayName and GetQualityColor)
-

Key Design Decisions

Switch Statements vs Dictionary Lookup

The current implementation uses switch statements in `GetWeaponMaterial()` / `GetArmorMaterial()` rather than a `Dictionary<ItemMaterialType, MaterialStats>`. This is intentional because D&D material costs often depend on the base item's properties (e.g., silver cost varies by weapon handedness, adamantine armor cost varies by category). Switch cases allow clean per-material logic with item-dependent calculations.

Legacy Bypass Flags

`ItemData` retains `IsSilvered`, `IsColdIron`, `IsAdamantine` boolean fields alongside the new `Material.WeaponBypassTags` system. Both are checked in `GetBypassTags()`. This ensures backward compatibility with manually-defined NPC equipment while the new system handles all factory-generated items.

Masterwork Cost Centralization

All masterwork cost calculations go through `MaterialProperties.GetMasterworkCost(item)`:

- Weapons: 300gp (PHB p.126)
- Armor/Shields: 150gp (PHB p.126)
- Tools: 50gp

No hardcoded masterwork costs exist outside this method.

Runtime API

```
// Create a pre-registered variant (fastest – looks up from database)
ItemData adamLongsword = ItemDatabase.CloneItem("adamantine_longsword");

// Create a variant on the fly (returns new item, doesn't register)
ItemData mithralBreastplate = ItemMaterialFactory.CreateMaterialArmor(
    ItemDatabase.Get("breastplate"), ItemMaterialType.Mithral);

// Apply material to an existing item in-place (mutates)
ItemData loot = ItemDatabase.CloneItem("longsword");
ItemMaterialFactory.ApplyMaterial(loot, ItemMaterialType.ColdIron);

// CR-based random loot
ItemData randomWeapon = ItemMaterialFactory.GetRandomMaterialWeapon("longsword", cr:
8);
```